AI Reporting POC — Study Guide: Databases, SQL, Schema Graph and Schema RAG

Oct 2, 2026 · @Mr. Asghar

How to use this guide

Spread this over three days of one to two hours each, with SSMS open beside you. Every idea here uses the AdventureWorks database you restored, so you can check it yourself as you read.

Day	Read	Practise
1	Part 1 (database fundamentals) and Part 2 (tour of AdventureWorks)	Browse the tables in SSMS; build a database diagram of the sales tables
2	Part 3 (SQL essentials)	Do all the exercises before looking at the solutions
3	Part 4 (what we built), Part 5 (schema RAG vs schema graph), Part 6 (remaining pieces)	Rerun both scripts, open schema.json, then answer the self-check questions

Don't rush Day 1 and Day 2. The AI parts only make sense once tables, keys and joins feel familiar, because the AI's whole job is to write SQL against them.

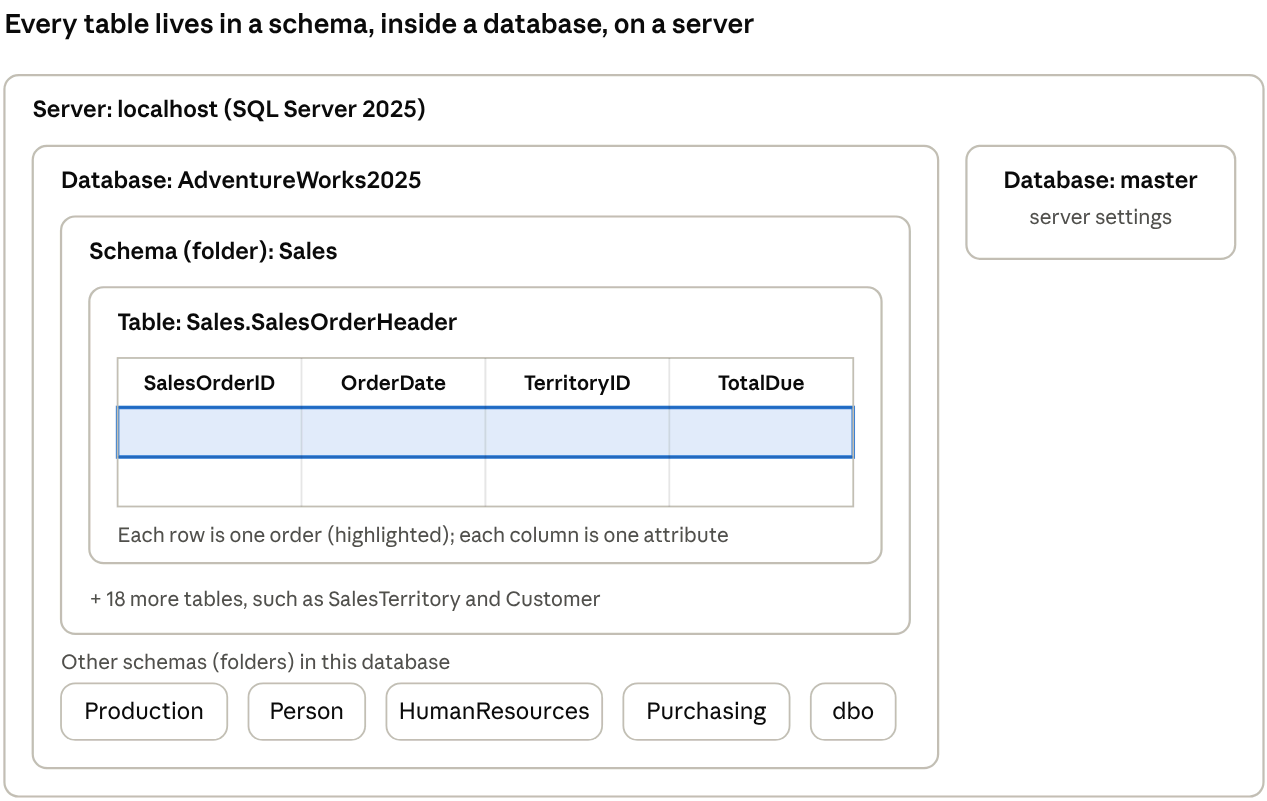
Part 1: Database fundamentals

A database stores data in tables, and tables link to each other through keys. Everything in this project, including the AI, builds on those two ideas.

The building blocks, from biggest to smallest

1. **Server (instance).** The running SQL Server program on your laptop, which you connect to as `localhost`. One server can hold many databases.
2. **Database.** A separate container for one system's data, such as `AdventureWorks2025`. The `master` database you saw in SSMS is SQL Server's own settings database.

3. **Schema (folder).** A named group of tables inside a database, usually one per department: `Sales`, `Production`, `HumanResources`.
4. **Table.** One kind of thing: orders, products, employees. Its full name includes its folder, as in `Sales.SalesOrderHeader`.
5. **Row.** One instance of that thing: one order.
6. **Column.** One attribute every row has: `OrderDate`, `TotalDue`.



SQL Server hierarchy · server, database, schema, table

Read it from the outside in: the server holds databases, each database holds schemas, and each schema holds tables made of rows and columns.

If you know Excel: a database is like a workbook, a table is like a sheet, and rows and columns mean the same thing. The differences are what make databases reliable. Every column has a fixed data type, rows are linked by keys, tables can hold billions of rows, and many people can read and write at the same time.

The word "schema" has two meanings

This trips up almost everyone. You will meet both meanings in this project.

Meaning	What it is	Example
Database schema (blueprint)	The full structure of a database: every table, column, data type and key, but no actual data	`schema.json`, "schema RAG", "schema graph"
SQL Server schema (folder)	A named group of tables inside one database	`Sales`, `Person`, `Production`

When this guide says "the schema" on its own, it means the blueprint. When it means the folder, it names it, like "the Sales schema".

Data types

Every column accepts only one type of value. A date column cannot hold the word "tomorrow", and a number column cannot hold "N/A".

Type	Holds	AdventureWorks example
`int`	Whole numbers	`SalesOrderID`
`nvarchar`	Text (any language)	`Production.Product.Name`
`datetime`	Date and time	`SalesOrderHeader.OrderDate`
`money`	Currency amounts	`SalesOrderHeader.TotalDue`
`decimal`	Exact decimal numbers	`Production.Product.Weight`
`bit`	Yes or no, stored as 1 or 0	`SalesOrderHeader.OnlineOrderFlag`

Types matter for the AI too. A `datetime` column tells it to filter with a date range, while an `nvarchar` column tells it to match text.

Primary keys: one ID per row

A primary key is a column whose value is unique for every row and never empty. In `Sales.SalesOrderHeader` it is `SalesOrderID`, so order 43659 means exactly one order.

Sometimes two columns together form the key, called a **composite key**. `Sales.SalesOrderDetail` uses `SalesOrderID` plus `SalesOrderDetailID`, because one order has several lines.

Foreign keys: links between tables

A foreign key is a column that holds another table's primary key value. `SalesOrderHeader.TerritoryID = 4` means "this order belongs to the territory whose ID is 4 in `Sales.SalesTerritory`", which is Southwest.

Why not just write "Southwest" on every order? Because there are about 31,000 orders. If the territory were renamed, you would change 31,000 rows instead of one, and someone would eventually type "South-west" and break every report. Storing each fact once and linking to it is called **normalization**.

The database also enforces the link. It refuses an order with `TerritoryID = 99` if no territory 99 exists. This guarantee is called **referential integrity**.

Three kinds of relationships

1. **One-to-many** (the most common). One territory has many orders, but each order has one territory. The foreign key always sits on the "many" side, so `TerritoryID` lives in the orders table.
2. **Many-to-many.** One order contains many products, and one product appears in many orders. Two tables cannot express this directly, so a **bridge table** sits between them. `Sales.SalesOrderDetail` is that bridge: each row is one product on one order.
3. **One-to-one.** Each employee is exactly one person. `HumanResources.Employee` and `Person.Person` share the same ID, called `BusinessEntityID`. AdventureWorks gives every party (person, store, vendor) a `BusinessEntityID` in `Person.BusinessEntity`, which is why that column name appears in so many tables.

Metadata: data about the data

A database also describes itself. Built-in views like `INFORMATION_SCHEMA.TABLES` and `INFORMATION_SCHEMA.COLUMNS` list every table and column, and `sys.foreign_keys` lists every link. Descriptions written by people are stored as **extended properties** named `MS_Description`.

This is how our system learns the schema without anyone typing it in: `extract_schema.py` reads exactly these views.

Part 2: A guided tour of AdventureWorks

Adventure Works Cycles is a fictional company that makes and sells bicycles, parts, clothing and accessories. Its database has 71 tables in six SQL Server schemas, matching the departments of a real company.

Schema	Tables	What it covers	Tables to know
Sales	19	Orders, customers, territories, salespeople, special offers	`SalesOrderHeader`, `SalesOrderDetail`, `Customer`, `SalesTerritory`, `SalesPerson`
Production	25	Products, categories, inventory, manufacturing	`Product`, `ProductSubcategory`, `ProductCategory`
Person	13	People, addresses, contact details	`Person`, `Address`, `BusinessEntity`
HumanResources	6	Employees, departments, pay and job history	`Employee`, `Department`, `EmployeeDepartmentHistory`
Purchasing	5	Buying from suppliers	`PurchaseOrderHeader`, `Vendor`
dbo	3	Housekeeping logs, not business data	none

Following one sale through the tables

The easiest way to understand a database is to follow one real event through it. Here is what one sale touches:

1. A customer places an order. One row is added to `Sales.SalesOrderHeader`, holding the date, the customer, the territory, the salesperson and the totals.
2. The order contains three products, so three rows are added to `Sales.SalesOrderDetail`, one per product line, with quantity, unit price and `LineTotal`.
3. Each order line points to a product in `Production.Product`. Each product points to a subcategory (Mountain Bikes), and each subcategory points to a category (Bikes).

4. The header points to `Sales.SalesTerritory` for the region, and to `Sales.SalesPerson` for the rep. The rep is also an employee in `HumanResources.Employee`, who is also a person in `Person.Person`.
5. The header points twice to `Person.Address`: once for the billing address, once for the shipping address.

The header and detail pattern

Splitting an order into one header row and many detail rows is very common. Invoices, purchase orders and delivery notes in your company's systems almost certainly work the same way.

It also creates the classic reporting trap. "Total sales" could mean either of these, and they give different numbers:

- `SUM(SalesOrderHeader.TotalDue)` includes tax and freight.
- `SUM(SalesOrderDetail.LineTotal)` is product revenue only.

Neither is wrong. It's a business definition someone must decide once, and the system must remember. Part 6 covers how.

Exploring it yourself in SSMS

- Expand any table in Object Explorer, then its **Columns** and **Keys** folders. Primary keys show a gold key icon; foreign keys are listed under Keys with names starting `FK_`.
- Right-click a table and choose **Select Top 1000 Rows** to see real data without writing SQL.
- Right-click **Database Diagrams** under `AdventureWorks2025` and choose **New Database Diagram**. Add `SalesOrderHeader`, `SalesOrderDetail`, `Product`, `ProductSubcategory`, `ProductCategory` and `SalesTerritory`. SSMS draws the foreign-key lines between them. If it asks to install diagram support, click **Yes**.

Part 3: SQL essentials

Seven ideas cover almost every report query: SELECT, WHERE, ORDER BY, aggregates, GROUP BY, JOIN and date filters. Run every example in SSMS with `AdventureWorks2025` selected in the toolbar dropdown.

First, find which years your copy of the data covers, since some examples filter by year:

```
SELECT YEAR(OrderDate) AS OrderYear, COUNT(*) AS Orders
FROM Sales.SalesOrderHeader
GROUP BY YEAR(OrderDate)
ORDER BY OrderYear;
```

Wherever an example uses 2013, swap in a year from your result.

1. SELECT and FROM: which columns, from which table

```
SELECT TOP 5 p.Name AS ProductName, p.ListPrice AS Price
FROM Production.Product AS p;
```

`TOP 5` limits the result to five rows. `AS p` gives the table a short nickname (an **alias**), and `AS ProductName` renames a column in the result. `SELECT *` means "all columns", which is fine for exploring but not for reports.

2. WHERE: which rows

```
SELECT Name, Color, ListPrice
FROM Production.Product
WHERE Color = 'Red' AND ListPrice > 1000;
```

Operator	Meaning	Example
`=`, `<>`	Equal, not equal	`Color = 'Red'`
`>`, `<`, `>=`, `<=`	Comparisons	`ListPrice > 1000`
`IN`	Any of a list	`Color IN ('Red', 'Black')`
`LIKE`	Text pattern; `%` means anything	`Name LIKE 'Mountain%'`
`IS NULL`	Value is missing	`Color IS NULL`
`AND`, `OR`	Combine conditions	`A AND (B OR C)`

NULL means "no value". It is not zero and not empty text. You must test it with `IS NULL`, never `= NULL`.

3. ORDER BY: sorting

Add `ORDER BY ListPrice DESC` to sort highest first. `ASC` (the default) sorts lowest first. Without ORDER BY, SQL Server returns rows in no guaranteed order.

4. Aggregates and GROUP BY: summarising

Aggregates turn many rows into one number: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. GROUP BY does that once per group.

```
SELECT Color, COUNT(*) AS Products, AVG(ListPrice) AS AvgPrice
FROM Production.Product
WHERE Color IS NOT NULL
GROUP BY Color
HAVING COUNT(*) > 20
ORDER BY Products DESC;
```

The rule to remember: every column in SELECT must either appear in GROUP BY or sit inside an aggregate. **HAVING** filters groups after they are summarised, while WHERE filters rows before.

5. JOIN: combining tables

A join matches rows from two tables wherever a foreign key equals a primary key.

```
SELECT p.Name AS Product, ps.Name AS Subcategory
FROM Production.Product AS p
JOIN Production.ProductSubcategory AS ps
  ON ps.ProductSubcategoryID = p.ProductSubcategoryID;
```

- **JOIN** (also called INNER JOIN) keeps only rows that match on both sides. Products with no subcategory, like loose nuts and bolts, disappear.
- **LEFT JOIN** keeps every row from the left table. Where nothing matches, the right side's columns come back as NULL.

Change `JOIN` to `LEFT JOIN` above and compare the row counts in the bottom-right corner of SSMS. That difference is one of the most common causes of wrong report totals.

6. Date filters

```
SELECT COUNT(*) AS Orders2013
FROM Sales.SalesOrderHeader
WHERE OrderDate >= '2013-01-01' AND OrderDate < '2014-01-01';
```

Use "on or after the start, before the next start". `BETWEEN '2013-01-01' AND '2013-12-31'` can silently miss orders placed on 31 December after midnight, because `datetime` columns carry a time. `YEAR(OrderDate)` and `MONTH(OrderDate)` pull out parts of a date for grouping.

7. The order SQL actually runs in

You write SELECT first, but the database works in this order. Knowing it explains most error messages:

1. `FROM` and `JOIN`: gather the tables
2. `WHERE`: drop rows
3. `GROUP BY`: form groups
4. `HAVING`: drop groups
5. `SELECT`: pick and calculate columns
6. `ORDER BY`: sort

This is why you cannot use a SELECT alias inside WHERE: WHERE runs before SELECT has named it.

Exercises

Try each one before reading its solution.

1. List every product with "Helmet" in its name, with its price.
2. Show the number of orders and the total `TotalDue` for each year.
3. How many orders were placed online versus not? (Hint: `OnlineOrderFlag`.)
4. Show total sales for each territory group: North America, Europe and Pacific. (Hint: `SalesTerritory` has a column called `Group`, which is also a SQL keyword, so write it as `[Group]`.)
5. Show the five individual customers who spent the most, with their first and last names.
6. List every salesperson with their number of orders, including anyone with zero.

7. Compare `SUM(TotalDue)` from the headers with `SUM(LineTotal)` from the details. Why do they differ?

Solutions

```
-- 1
SELECT Name, ListPrice FROM Production.Product
WHERE Name LIKE '%Helmet%';

-- 2
SELECT YEAR(OrderDate) AS OrderYear, COUNT(*) AS Orders, SUM(TotalDue) AS
Sales
FROM Sales.SalesOrderHeader
GROUP BY YEAR(OrderDate)
ORDER BY OrderYear;

-- 3
SELECT OnlineOrderFlag, COUNT(*) AS Orders
FROM Sales.SalesOrderHeader
GROUP BY OnlineOrderFlag;

-- 4
SELECT st.[Group] AS Region, SUM(soh.TotalDue) AS Sales
FROM Sales.SalesOrderHeader AS soh
JOIN Sales.SalesTerritory AS st ON st.TerritoryID = soh.TerritoryID
GROUP BY st.[Group]
ORDER BY Sales DESC;

-- 5
SELECT TOP 5 p.FirstName, p.LastName, SUM(soh.TotalDue) AS Spent
FROM Sales.SalesOrderHeader AS soh
JOIN Sales.Customer AS c ON c.CustomerID = soh.CustomerID
JOIN Person.Person AS p ON p.BusinessEntityID = c.PersonID
GROUP BY p.FirstName, p.LastName
ORDER BY Spent DESC;

-- 6
SELECT p.FirstName, p.LastName, COUNT(soh.SalesOrderID) AS Orders
FROM Sales.SalesPerson AS sp
JOIN Person.Person AS p ON p.BusinessEntityID = sp.BusinessEntityID
LEFT JOIN Sales.SalesOrderHeader AS soh ON soh.SalesPersonID =
sp.BusinessEntityID
```

```
GROUP BY p.FirstName, p.LastName
ORDER BY Orders DESC;

-- 7
SELECT
  (SELECT SUM(TotalDue)  FROM Sales.SalesOrderHeader) AS HeaderTotalDue,
  (SELECT SUM(LineTotal) FROM Sales.SalesOrderDetail) AS DetailLineTotal;
```

Notes on the solutions:

- **5** only covers individual customers. Store customers have an empty `PersonID`, so the inner join to `Person.Person` drops them. Whether that's right depends on what "customer" means in the report.
- **6** uses LEFT JOIN so salespeople with no orders stay in the list, and `COUNT(soh.SalesOrderID)` counts only real orders, giving them 0. `COUNT(*)` would wrongly give them 1.
- **7** differs because `TotalDue` adds tax and freight on top of product revenue.

Part 4: What we built so far

Two pieces of the AI system already work: a schema extractor that reads the database's blueprint, and a schema graph that finds how any set of tables joins. Neither uses AI yet; they prepare what the AI will read.

File	What it does	When it runs
`db.py`	Holds the connection settings in one place	Imported by every other script
`test_connection.py`	Proves Python can reach SQL Server	Once, during setup
`extract_schema.py`	Reads tables, columns, keys and descriptions from the database	Once at setup, then again whenever the database changes
`schema.json`	The saved blueprint: 71 tables, 486 columns, 91 foreign-key links	Read by later phases
`schema_graph.py`	Builds the table graph and finds join paths	Whenever join paths are needed

File	What it does	When it runs
`schema_graph.json`	The saved graph	Loaded by later phases

How Python talks to SQL Server

Python cannot speak to SQL Server directly. It goes through a chain: your script calls **SQLAlchemy** (manages connections), which calls **pyodbc** (Python's ODBC bridge), which uses the **ODBC Driver 18** (Microsoft's translator), which talks to SQL Server.

The connection string in `db.py` is the SSMS login window written as text: server `localhost`, database `AdventureWorks2025`, Windows Authentication (`trusted_connection=yes`), and the "Trust server certificate" tick (`TrustServerCertificate=yes`).

extract_schema.py, step by step

1. It runs four metadata queries: one for columns and their types, one for primary keys, one for foreign keys, and one for the `MS_Description` texts.
2. It loads each result into a pandas DataFrame, which is a table in memory.
3. It merges them into one entry per table, with the table's description, its columns, its primary key and its foreign keys.
4. It saves everything to `schema.json` and prints a summary.

One table in `schema.json` has this shape (shortened):

```
{
  "name": "Sales.SalesOrderHeader",
  "description": "General sales order information.",
  "columns": [
    {"name": "SalesOrderID", "type": "int", "nullable": false, "description":
"..."},
    {"name": "OrderDate", "type": "datetime", "nullable": false,
"description": "..."}
  ],
  "primary_key": ["SalesOrderID"],
  "foreign_keys": [
    {"column": "TerritoryID", "references_table": "Sales.SalesTerritory",
"references_column": "TerritoryID"}
  ]
}
```

The `...` stand for the description text stored in the database; open your own `schema.json` to read the real ones. 69 of 71 tables came with a description. Real company databases usually have almost none, so later the AI will draft them and your team will correct them.

What a graph is

In computing, a **graph** is not a chart. It is a set of **nodes** (things) joined by **edges** (connections), like stations and lines on a metro map. In our schema graph, each table is a node and each foreign key is an edge.

schema_graph.py, step by step

1. It loads `schema.json` and adds every table as a node, using the **NetworkX** library.
2. For each foreign key, it adds an edge between the two tables and stores the join condition on it, such as `SalesOrderHeader.TerritoryID = SalesTerritory.TerritoryID`.
3. When a foreign key spans two columns (a composite key), it combines them into one condition with AND.
4. When two tables are linked more than once, it keeps all the conditions on one edge. That's why 91 foreign keys became 86 edges.

5. Given a list of tables, it finds the **Steiner tree**: the smallest set of edges connecting all of them, adding in-between tables where needed. With two tables, that's simply the shortest path.
6. It walks that tree from the first table and prints the joins in order.

The bug, and what it taught

The first run crashed on `dbo.AWBuildVersion`. Four tables have no foreign keys at all, so they sit on their own "islands" of the graph. The Steiner algorithm assumed every table could be reached and failed when one couldn't.

The fix keeps only the **connected component**, meaning the island of tables reachable from the ones requested. Unlinked tables can never be part of a join, so ignoring them loses nothing.

Reading the results

Question	Path found	Lesson
Sales by territory	1 join, direct	Easy cases need no help
Sales by product category	5 joins through 4 extra tables	The graph finds in-between tables nobody named
Department to sales	4 joins across HR and Sales	Joins come from the graph; business rules (current department only) do not
Sales by city	1 join, flagged as having 2 options	The graph detects ambiguity (billing or shipping address) but cannot resolve it

The category path went through `Sales.SpecialOfferProduct` because the database never declared a direct link from order lines to products. A person would join `SalesOrderDetail.ProductID` straight to `Product`. Missing links like this are common in real databases, and we will let your team add them by hand as **virtual edges**.

The hub tables matter too. `Production.Product` has 16 connections, more than any other table. In most companies, a few central tables like products, customers and employees connect every department, and many join paths run through them.

Part 5: Schema RAG vs schema graph

Schema RAG finds **which tables** a question is about, by meaning. The schema graph finds **how those tables connect**, by structure. The AI needs both answers before it can write correct SQL.

An analogy

You ask a city guide: "I want good coffee and a bookshop, then somewhere to watch the sunset." First the guide works out which places match your words: a particular café, a bookshop, a hilltop. That's schema RAG. Then the guide opens the road map to plan the route between them, including the streets you never mentioned. That's the schema graph.

The guide who only knows places can't get you there. The guide who only has the map doesn't know where you want to go.

How schema RAG works

RAG means Retrieval-Augmented Generation: first **retrieve** the few relevant pieces of knowledge, then let the AI **generate** its answer using only those.

1. **Once, at setup:** each table's description (and its column descriptions) is turned into an **embedding**, a list of numbers that captures its meaning. These are stored in a **vector database**.
2. **For each question:** the question is turned into an embedding the same way.
3. The system compares the question's numbers with every table's numbers using **cosine similarity**, a score from 0 (unrelated) to 1 (same meaning).
4. The highest-scoring tables are handed to the AI.

The point of embeddings is that similar meanings get similar numbers even when the words differ. "Revenue" scores close to a column described as "total amount due from the customer", though they share no words.

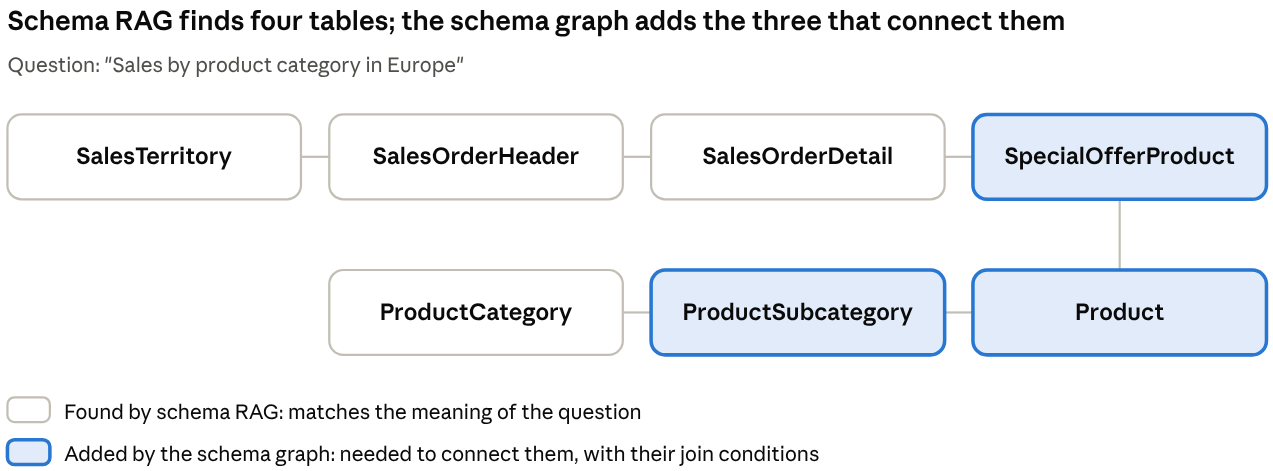
Side by side

	Schema RAG	Schema graph
Answers	Which tables is this question about?	How do these tables join?

	Schema RAG	Schema graph
Built from	Table and column descriptions	Foreign keys
Works by	Comparing meanings (embeddings)	Following links (paths)
Understands words like "revenue"	Yes	No
Knows join conditions	No	Yes
Finds in-between tables nobody mentioned	No	Yes
Spots two possible links (billing or shipping)	No	Yes
Our status	Next to build	Built

Worked example: "Sales by product category in Europe"

1. **Schema RAG** compares the question with all 69 described tables. Descriptions about orders, product categories and sales territories score highest, so it returns `SalesOrderHeader`, `SalesOrderDetail`, `ProductCategory` and `SalesTerritory`.
2. It does **not** return `Product`, `ProductSubcategory` or `SpecialOfferProduct`. Their descriptions aren't about sales, categories or Europe, so their scores stay low, yet the query cannot run without them.
3. **The schema graph** takes the four tables, finds the Steiner tree, adds the missing in-between tables, and returns every join condition.
4. **The AI** receives about seven tables plus their exact joins, instead of 71 tables, and writes the SQL. It still needs one more fact: that "Europe" is stored as `SalesTerritory.[Group] = 'Europe'`. That comes from data RAG, covered in Part 6.



Worked example · 7 tables, 6 joins

Each line is a join. The three highlighted tables never appear in the question, which is why meaning-based search misses them and the graph has to add them.

Why not use just one?

- **Graph only:** the graph has no idea that "sales" means `SalesOrderHeader`. It needs to be told which tables to connect.
- **RAG only:** the AI gets the right topic tables but has to guess the joins. On the category question it would very likely skip `ProductSubcategory`, and the query would fail or, worse, return wrong numbers.

Why not just give the AI all 71 tables?

Honestly, for AdventureWorks you could. 71 tables and 486 columns fit in a modern AI's context window. Schema RAG becomes essential at company scale, where thousands of tables and tens of thousands of columns won't fit, or fit but drown the AI in distractions.

Irrelevant tables also cause mistakes even when they fit. `Purchasing.PurchaseOrderHeader` and `Sales.SalesOrderHeader` both look like "orders", and an AI shown both may pick the wrong one. We build RAG on AdventureWorks so it is proven and ready when you point the system at real data.

Part 6: Where the remaining pieces fit

Schema RAG and the schema graph tell the AI where the data lives. Three more pieces tell it what the user's words mean, what the business has already decided, and whether the result is safe to run.

Data RAG: matching the user's words to real values

Users don't type values the way the database stores them. Someone asks for "Europe" and the database has `[Group] = 'Europe'`; someone asks for "road bikes" and the subcategory is called `Road Bikes`; someone asks for "completed orders" and `Status` holds the number 5.

If the AI writes `WHERE Status = 'completed'`, the query runs without error and returns nothing. Data RAG prevents this by storing the distinct values of descriptive columns (names, categories, regions, statuses) as embeddings, and matching the question's words against them.

It never embeds the millions of order rows. A table of 31,000 orders has only a handful of distinct territories and statuses, so the data knowledge base stays small.

Episodic memory: remembering decisions

Some questions have no answer in the database at all. These are business decisions:

- Does "sales" mean `TotalDue` (with tax and freight) or `LineTotal` (products only)?
- Does "sales by city" use the billing or the shipping address?
- Does "department" mean the employee's current department, or every department they've been in?

The first time, the system asks a person or shows its assumption for approval. Episodic memory stores the approved question, the SQL and the decision. Next time a similar question arrives, the system finds that past example and reuses the decision. This is also where saved reports live, so a recurring report is never regenerated from scratch.

The LLM: writing the SQL

The LLM (the AI model) receives a compact package: the question, the handful of relevant tables, their join conditions, the matching real values, and any similar past examples. Its only job is to write one SQL query from that.

Validation and execution: keeping it safe

Before anything runs, plain Python code checks the SQL. It must only read data (no `DELETE` or `UPDATE`), every table and column must exist, and the user must be allowed to see that data. The query then runs with a time limit, so a bad query can't slow the database for everyone.

One request, end to end

1. The user asks: "Sales by product category in Europe for 2013, as an Excel table."
2. Episodic memory finds a past approved example and the rule "sales means LineTotal".
3. Schema RAG returns the order, category and territory tables.
4. Data RAG maps "Europe" to `SalesTerritory.[Group] = 'Europe'`.
5. The schema graph adds the in-between tables and the join conditions.
6. The LLM writes the SQL.
7. Validation checks it; SQL Server runs it.
8. Python formats the few result rows into the requested Excel layout.

So far we have built the groundwork for step 3 (the schema extraction that schema RAG reads) and all of step 5. Next comes the rest of schema RAG, then data RAG, then the LLM step once the API key is ready.

Glossary

Term	Meaning
Aggregate	A function that turns many rows into one value: COUNT, SUM, AVG, MIN, MAX
Alias	A short nickname for a table or column inside a query, like `soh`
Bridge table	A table that links two others in a many-to-many relationship, like `SalesOrderDetail`
Composite key	A key made of two or more columns together
Connected component	A group of nodes in a graph that can all reach each other; an "island"

Term	Meaning
Cosine similarity	A score from 0 to 1 for how close two embeddings are in meaning
Database schema	The blueprint of a database: tables, columns, types, keys, no data
Edge	A connection between two nodes in a graph; for us, a foreign key
Embedding	A list of numbers that represents the meaning of a piece of text
Episodic memory	Stored past questions, approved SQL and business decisions, reused for similar questions
Foreign key	A column holding another table's primary key value, creating a link
Graph	A set of nodes joined by edges, like stations and lines on a metro map
Hub table	A table with many connections, like `Production.Product`
Join	Combining rows from two tables where a foreign key matches a primary key
LLM	Large language model: the AI that reads the package of context and writes SQL
Metadata	Data about the data: table names, column types, keys, descriptions
Node	One item in a graph; for us, a table
Normalization	Storing each fact once and linking to it, instead of repeating it
NULL	A missing value; not zero and not empty text
ODBC driver	Microsoft's translator that lets programs like Python talk to SQL Server
Primary key	A column whose value is unique for every row and never empty

Term	Meaning
RAG	Retrieval-Augmented Generation: retrieve the relevant knowledge first, then let the AI answer using it
Data RAG	RAG over real column values, so user wording maps to stored values
Schema RAG	RAG over table and column descriptions, to find which tables a question needs
Referential integrity	The database's guarantee that every foreign key points to a row that exists
SQL Server schema	A named folder of tables inside a database, like `Sales`
SSMS	SQL Server Management Studio, the desktop app for browsing and querying SQL Server
Steiner tree	The smallest set of edges that connects a chosen set of nodes, adding in-between nodes if needed
Vector database	A database that stores embeddings and finds the closest ones quickly
Virtual edge	A link your team adds by hand because the database never declared it

Self-check questions

Answer each in your own words before reading the answer under it. If you can answer all ten, you're ready for the next phase.

1. What is the difference between `Sales` and "the schema" of AdventureWorks?
   - `Sales` is a SQL Server schema: a folder of tables. "The schema" is the database's whole blueprint of tables, columns, types and keys.
2. Why does `SalesOrderHeader` store `TerritoryID` instead of the territory's name?
   - So the name is stored once. A rename changes one row, and typos can't create fake territories (normalization).
3. Where does the foreign key sit in a one-to-many relationship?

    - On the "many" side. Many orders point to one territory, so `TerritoryID` lives in the orders table.
4. Why does a LEFT JOIN sometimes change a report's total compared to a JOIN?
    - A JOIN drops rows with no match; a LEFT JOIN keeps them, with NULLs on the right side. Missing or extra rows change sums and counts.
5. Why is `SUM(TotalDue)` different from `SUM(LineTotal)`?
    - `TotalDue` includes tax and freight; `LineTotal` is product revenue only. Which one "sales" means is a business decision.
6. What does `extract_schema.py` read, and where does that information come from?
    - Tables, columns, types, primary keys, foreign keys and descriptions, all from the database's own metadata views and extended properties.
7. Why did 91 foreign keys become 86 edges?
    - Tables linked more than once (like billing and shipping addresses) share one edge holding several conditions, and composite keys count as one link.
8. What does the Steiner tree give us that a list of tables doesn't?
    - The in-between tables needed to connect them, and the exact join condition for each step.
9. In one sentence each: what does schema RAG do, and what does the schema graph do?
    - Schema RAG finds which tables a question is about by meaning. The schema graph finds how those tables join by following foreign keys.
10. Which piece would stop the AI from writing `WHERE [Group] = 'european countries'`?
    - Data RAG, which matches the user's words to the values actually stored, here `'Europe'`.